CLOUD / INFRA ENGINEER

김주우

Kim Joowoo

배포를 자동화하고, 실제 상태를 재고,
인프라를 코드와 권한으로 다루는 클라우드·인프라 엔지니어입니다.

Email kimzoo5676@gmail.com **GitHub** github.com/kimzoo5676-del **Blog** zoolog.site



이름 김주우

이메일 kimzoo5676@gmail.com

교육

2026.01 – 삼성 청년 SW·AI 아카데미(SSAFY) 15기
1학기 Python·Django, 2학기 Java·Spring

2019.03 – 2025.08 한양대학교(ERICA) 경제학부 졸업
계량경제학 · 경제데이터마이닝과머신러닝

자격

2026.03 데이터분석준전문가(ADsP)
한국데이터산업진흥원

Awards

2026.05–06 SSAFY RACE 자율주행 경진대회 2위
278팀 중 2위 · 삼성청년SW·AI아카데미

2026.06 SSAFY 1학기 관통 프로젝트 최우수상
OURWISH · 서울 2반 1등

Archive

GitHub github.com/kimzoo5676-del

Blog zoolog.site
직접 만들어 k3s에서 운영하는 학습 블로그

Web kimzoo5676-del.github.io

배포 자동화에서 코드와 권한까지

다섯 프로젝트에서 배포를 자동화하고, 배포가 성공으로 표시될 때도 실제 상태를 확인하며, **서버와 권한을 코드로 관리하는 데까지** 범위를 넓혔습니다.

2026.05 – 06

OURWISH

예·적금 추천 서비스

- 수동 배포를 GitHub Actions로
- 버튼을 눌러야 반영되는 배포
- AWS EC2 · Nginx

2026.07 –

MODI

협업 앱 · App Store 출시

- 테스트 통과 코드만 자동 배포
- blue/green 무중단 배포
- 출시 직전 OCI로 이관

2026.08 – 09

여신보좌관

기업 여신 심사 보조

- 팀 개발 환경을 Docker 이미지로
- develop 머지 자동 배포
- 배포 장애를 로그·파일로 진단

2026.10 –

Zoolog

1인 학습 블로그

- k3s + Argo CD GitOps
- 메모리 2GB 서버 한 대
- 화면 서버 무중단 교체

2026.10 – 진행 중

NEW:LEARN

AI 뉴스 분석 서비스

- Terraform · 최소 권한 IAM
- kubeadm 쿠버네티스
- CI 병목을 재고 러너 구성

배포 자동화

OURWISH · MODI · Zoolog — 손으로 하던 배포를 파이프라인으로, 테스트를 통과한 코드만 나가게

상태 확인

여신보좌관 · NEW:LEARN — 성공 표시가 아니라 로그와 지표로

코드와 권한

NEW:LEARN — Terraform으로 정의한 서버, 범위를 좁힌 IAM 정책

PROJECTS

PROJECTS

NEW:LEARN · MODI · 여신보좌관 · Zoolog · OURWISH

Main Projects & Tech Skills

2026.10 – 진행 중 · 인프라 담당

AI로 뉴스를 구조화하는 이슈 분석 서비스 – NEW:LEARN

Terraform · Kubernetes(kubeadm) · GitLab CI · Prometheus · Grafana

2026.07 – 운영 중 · Backend · Infra · AI

할 일·일정·자료를 함께 관리하는 협업 앱 – MODI

OCI(ARM) · GitHub Actions · Docker · Caddy · Spring Boot · App Store 출시

2026.08 – 2026.09 · Infra · 인증 · 감사 로그

기업 여신 심사를 돕는 AI 서비스 – 여신보좌관

Docker Compose · Swarm · Jenkins · GitLab CI · Nginx · Spring Boot

2026.10 – 운영 중 · 1인 개발

공부 글을 그래프로 쌓는 학습 블로그 – Zoolog

AWS Lightsail · k3s · Argo CD · GitHub Actions · GHCR

2026.05 – 2026.06 · Backend · Infra · AI · 최우수상

사회초년생을 위한 예·적금 추천 서비스 – OURWISH

Django · AWS EC2 · Nginx · GitHub Actions

CLOUD

AWS EC2 · IAM · S3 · Lightsail · Budgets · Oracle Cloud(OCI)

CONTAINER · ORCHESTRATION

Docker · Docker Compose · Docker Swarm · Kubernetes(kubeadm) · k3s · Helm · Argo CD · Cilium

CI / CD

GitHub Actions · GitLab CI · Jenkins

IAC · NETWORK

Terraform · Nginx · Caddy · Tailscale

OBSERVABILITY

Prometheus · Grafana · Loki

BACKEND · DATA

Spring Boot · Django · FastAPI · PostgreSQL · Redis · MinIO

AI · TOOLS

Claude Code · OpenAI API · Git

PROJECT 01

NEW:LEARN

AI로 뉴스를 구조화하는 이슈 분석 서비스

2026.10 – 진행 중 · SSAFY 2학기 팀 프로젝트



PROJECT 01 — **NEW:LEARN**

AI로 뉴스를 구조화하는 이슈 분석 서비스

2026.10 – 진행 중 · 최종 발표 11.18

SSAFY 2학기 팀 프로젝트 · 인프라 담당

설명

뉴스 기사를 AI로 마인드맵으로 구조화하고, 같은 사건을 다룬 기사를 묶어 이슈의 흐름으로 보여 주는 서비스

역할

인프라 전담 · 백엔드 공동 — 서버 구축, 클러스터, CI, 모니터링

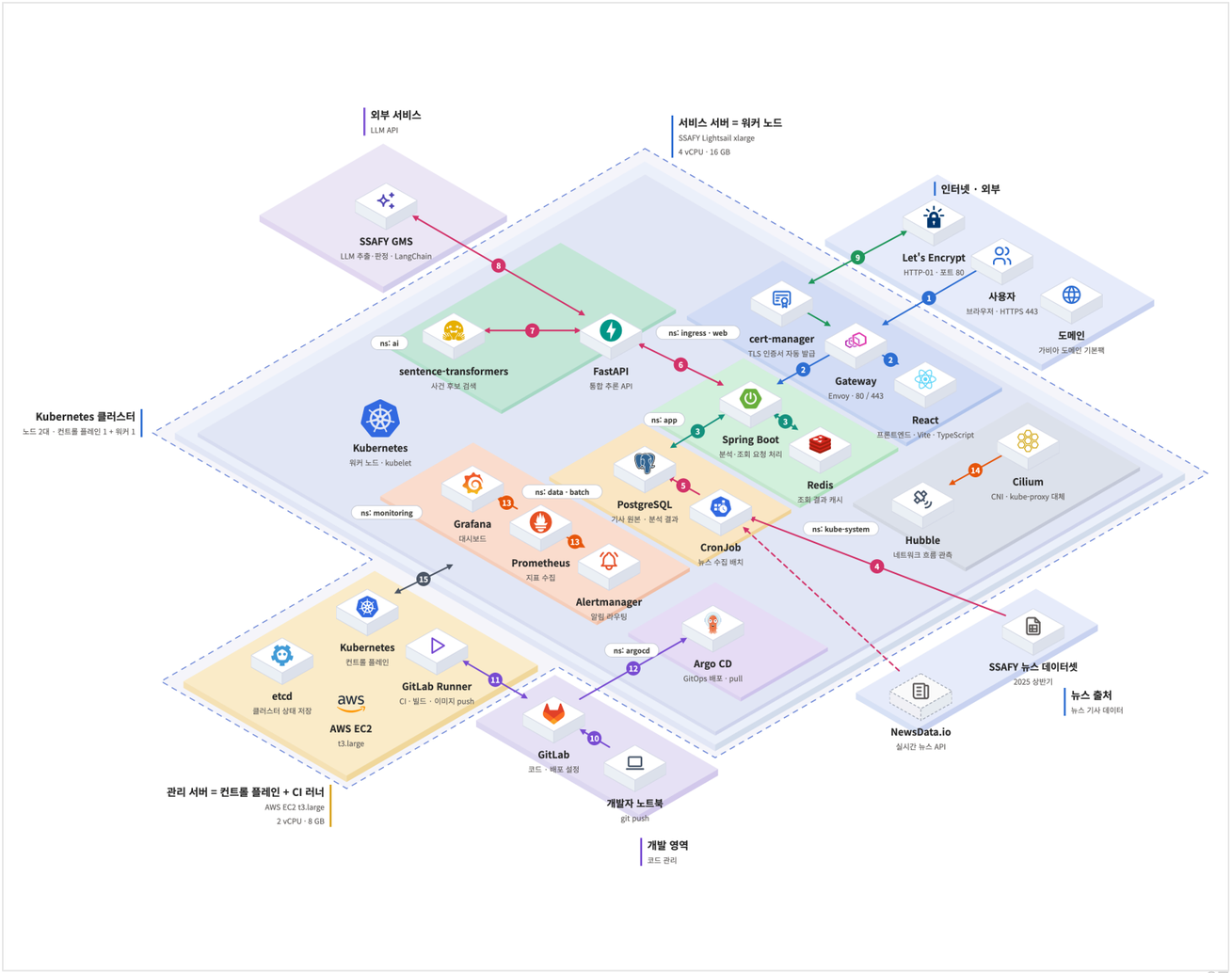
305→119초	**136→39초**	**4 / 476**
MR 파이프라인 러너 CPU 재배치 후	backend 잡 노트북 러너 합류 후	apply 키가 쓸 수 있는 AWS 서비스 수

주요 구현

- 서버 · 보안 그룹 · 고정 IP · 예산 알림을 Terraform으로 정의하고, 사람이 읽고 저장한 plan 파일로만 apply
- kubeadm + Cilium 컨트롤 플레인 위에 GitLab Runner(Kubernetes executor)를 올리고 잡 갈래별 CPU 요청 · 상한 지정
- CI를 만들기 전에 Prometheus · Grafana로 CPU · 스로틀부터 관측
- 실서버 전에 로컬 kind 클러스터에서 Argo CD 배포 · 되돌리기를 리허설(3회 모두 요청 실패 0)

> 관리 서버(EC2 · 컨트롤 플레인 · 러너 · 모니터링)까지 구축했고, 워커 노드(SSAFY Lightsail)와 Argo CD 배포는 서버를 받는 대로 붙입니다. 오른쪽은 목표 구성입니다.

AWS EC2 | Terraform | IAM | Kubernetes(kubeadm) | Cilium | GitLab CI | Helm | Prometheus | Grafana



PROJECT 01 — NEW:LEARN

CI를 만들기 전에 병목부터 재기

vCPU 2개 관리 서버에서
파이프라인 305초 → 119초

러너 CPU를 잡 갈래별로 나누기

backend 잡 136 → 39초

상황 여신보좌관은 CPU 4개인 운영 서버에 러너를 함께 두어, 테스트가 쌓이자 MR 파이프라인이 중앙값 약 11분, 기다림 상위 10% 23분까지 늘었습니다. NEW:LEARN 관리 서버는 vCPU 2개를 컨트롤 플레인 · 모니터링과 나눠 씁니다

설계 노드 예약 1190m를 빼고 남은 810m에 맞춰 갈래(W0~W4)별 CPU 요청 · 상한을 정했습니다 — backend 잡 550m

측정 잡 끝에 `cpu.stat`을 남겨 backend 잡이 2221 구간 중 2036(92%)에서 상한에 막힌 것을 확인했습니다

해결 CPU 4개인 노트북을 두 번째 러너로 붙이고, EC2 러너는 늘 켜진 기본 러너로 두었습니다

결과 **backend 잡 136초 → 39초, 파이프라인 305초 → 119초**

관측을 먼저 올리고, 캐시는 재서 고르기

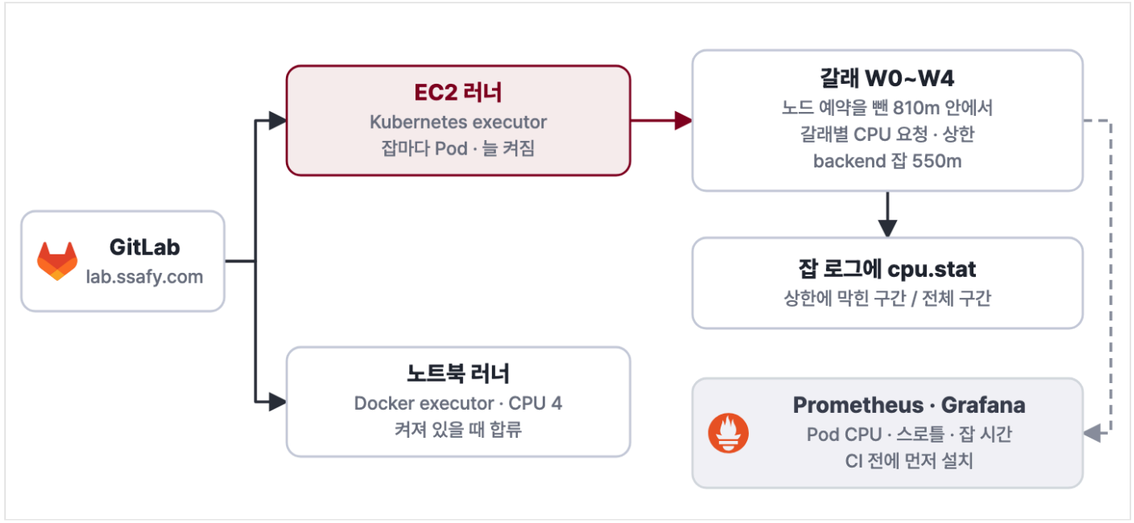
캐시 572초 → 320초

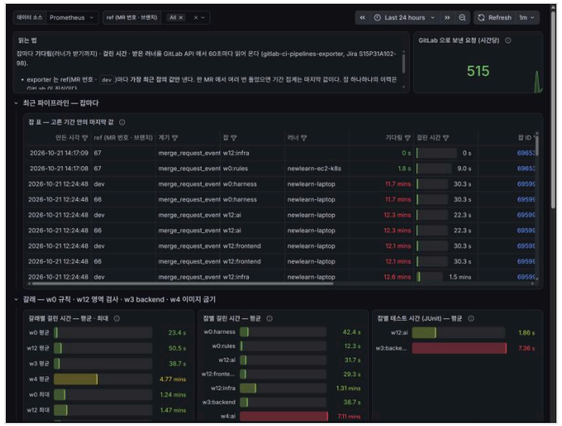
관측 추가 요금이 없는 Prometheus · Grafana를 CI보다 먼저 올리고, CPU 상한 0.1 시험 Pod로 스로틀 지표가 잡히는지 확인했습니다(98.7%)

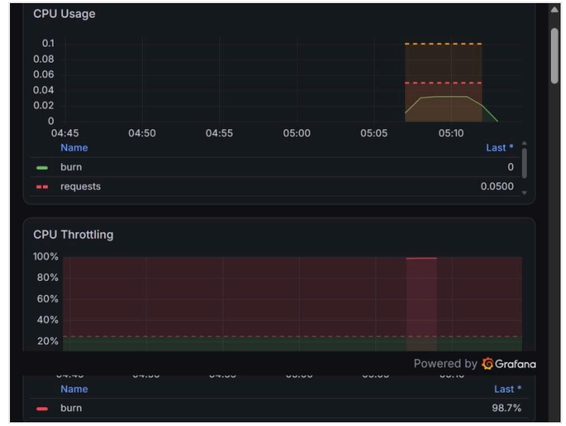
캐시 캐시 없이 480~490초, GitLab 기본 캐시는 압축 때문에 오히려 572초 → 캐시 폴더를 직접 쓰는 방식으로 바꿔 320초

결과 **CI를 21번 돌린 뒤에도 CPU 크레딧 잔액 54 → 269.8 — 지금 서버로 감당합니다**

> 구현과 측정은 Claude Code에 맡기고, 구성과 비용이 걸린 결정은 직접 했습니다. 잰 방법과 함께 기록된 값만 근거로 씁니다.







PROJECT 01 — NEW:LEARN

공부한 IAM 원칙을 Terraform 서버 구축에

콘솔 대신 코드로
apply 권한 476개 서비스 중 4개

콘솔 클릭 대신 코드로 서버 만들기

자원 7개 · apply 19초

상황 콘솔에서 만든 자원은 무엇이 만들어졌는지 코드로 남지 않아, 다시 만들거나 지울 때 사람 기억에 기댑니다

해결 서버 · 보안 그룹 · 고정 IP · 예산 알림을 Terraform으로 정의했습니다. 상태 파일은 S3(버전 관리 · 퍼블릭 차단 · 잠금)에 두고, plan을 읽은 뒤 저장한 plan 파일로만 apply합니다

방어 SSH는 관리자 IP /32만 열고 `0.0.0.0/0` 입력은 변수 검사로 거절하며, 보안 그룹은 테스트로 검사합니다. IMDSv2 필수, 예산 알림 80 · 100% · 예측

결과 **plan으로 확인한 자원 7개를 그대로 apply했고, 같은 코드로 다시 만들고 지울 수 있습니다**

apply 키의 권한을 필요한 만큼으로

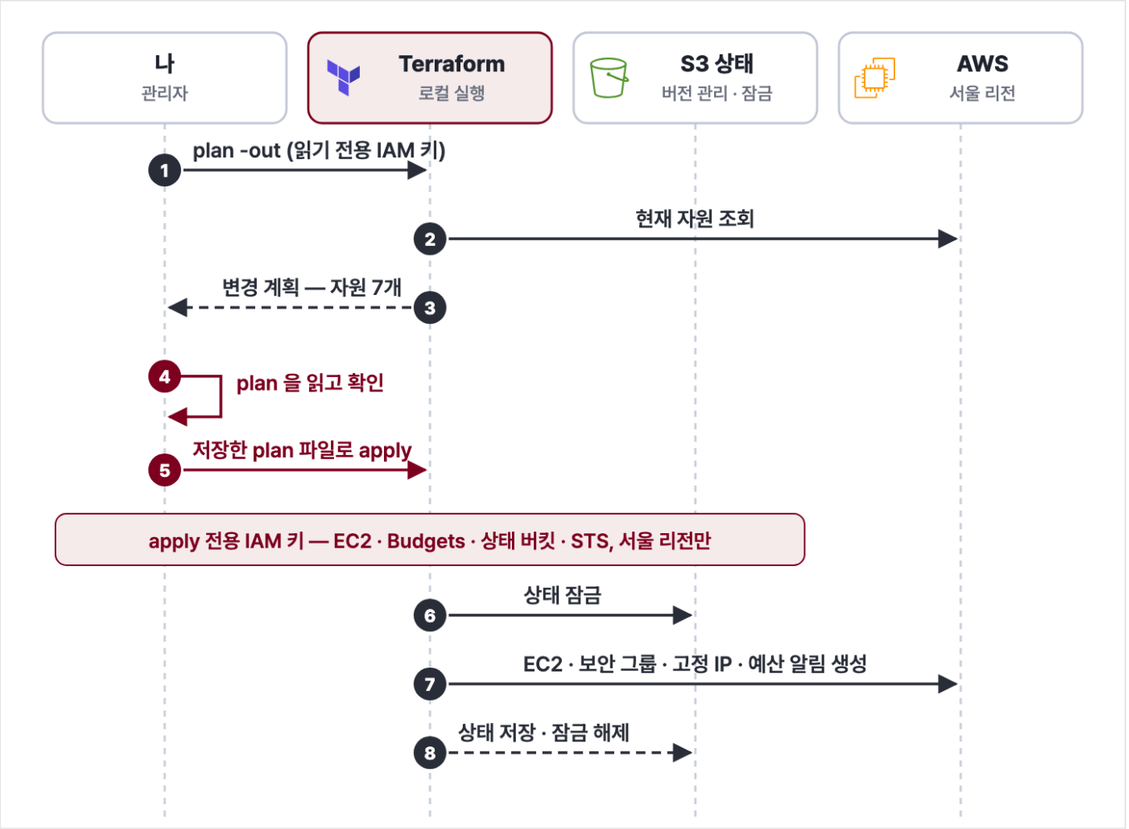
키가 새도 서울 리전 밖은 불가

상황 넓은 권한의 키는 새는 순간 계정 전체가 위험합니다. Zoolog에 정리한 IAM 공부(최소 권한 · Condition 키 · 루트 키 삭제)를 그대로 적용했습니다

해결 루트 계정에는 액세스 키를 만들지 않고, plan은 읽기 전용 사용자, apply는 콘솔 접근 없는 전용 사용자로 나눴습니다. 고객 관리형 정책은 EC2 · Budgets · S3(`newlearn-tfstate-*` 만) · STS, EC2에는 `aws:RequestedRegion = ap-northeast-2`

판단 SSO(Identity Center)는 조직을 만들면 무료 크레딧이 사라져 쓰지 않고, 대신 장기 키의 범위를 좁혔습니다

결과 **다른 리전 조회가 거절되는 것까지 확인 — 키가 새도 서울 리전의 EC2 · 예산 · 상태 버킷 밖은 건드릴 수 없습니다**





PROJECT 02

MODI

할 일 · 일정 · 자료를 함께 관리하는 협업 앱

2026.07 – 운영 중 · App Store 출시



PROJECT 02 — **MODI**

할 일 · 일정 · 자료를 함께 관리하는 협업 앱

2026.07 – 운영 중 · 4인

App Store 출시 「모두의 디데이 MODI」

역할

인프라 · 배포 · AI 서버 — 서버 이관, CI/CD, 무중단 배포, 백업, iOS 배포 빌드

08.19	**5xx 0건**	**20개 표**
2026년 App Store 출시 서버 이관을 끝내고	배포 중 보낸 요청 1,195건 (blue/green)	백업 복구 후 행 수 일치 확인

주요 구현

- 출시 직전 EC2 · GitLab · Jenkins에서 Oracle Cloud(ARM) · GitHub · Actions로 이관하고, 소스를 서버로 보내 ARM에서 네이티브 빌드
- 앱 · 서버 · AI 테스트 중 하나라도 실패하면 막히는 gate를 병합 조건으로
- Caddy upstream을 바꾸는 blue/green 무중단 배포
- 매일 04:00 DB 덤프 · 파일 저장소 백업, 임시 컨테이너에 복구해 20개 테이블 행 수 일치 확인
- 배포 빌드의 서명 · 권한(entitlements)을 빌드 스크립트가 검사해, 어긋나면 IPA를 내주지 않음

Oracle Cloud(ARM) | GitHub Actions | Docker Compose | Caddy | Spring Boot | FastAPI | PostgreSQL | Redis | MinIO

github.com/intpsquad/modi App Store 「모두의 디데이 MODI」



PROJECT 02 — MODI

출시 직전 서버 이관 — ARM 서버에서 빌드하기

2026.08.13 이관

2026.08.19 App Store 출시

x86 러너가 만든 이미지가 ARM 서버에서 돌지 않음

레지스트리 없이 배포

상황 기존 서버와 GitLab · Jenkins를 더 쓸 수 없게 되어, App Store 출시 전에 서버 · 저장소 · 배포 도구를 전부 옮겨야 했습니다

원인 새 서버는 Oracle Cloud Ampere A1(ARM64), GitHub Actions 러너는 x86_64라 러너에서 만든 이미지가 서버에서 뜨지 않습니다

해결 QEMU 에뮬레이션으로 ARM 이미지를 만드는 대신, 러너가 소스를 `rsync` 로 서버에 보내고 서버가 ARM에서 직접 빌드하게 했습니다

보안 배포 전용 SSH 키를 따로 만들고 서버 지문은 known_hosts에 고정했습니다. 서버에는 GitHub 자격 증명을 두지 않고, 비밀값 · 활성 색 기록은 저장소 밖 폴더에 두어 배포가 덮지 않게 했습니다

새 서버에 rsync가 없어 배포가 멈춤

배포 전 검사로

상황 2026.08.13 배포가 `rsync error: remote command not found (code 127)` 로 실패했습니다

원인 Oracle Cloud의 Ubuntu 최소 이미지에는 rsync가 없습니다

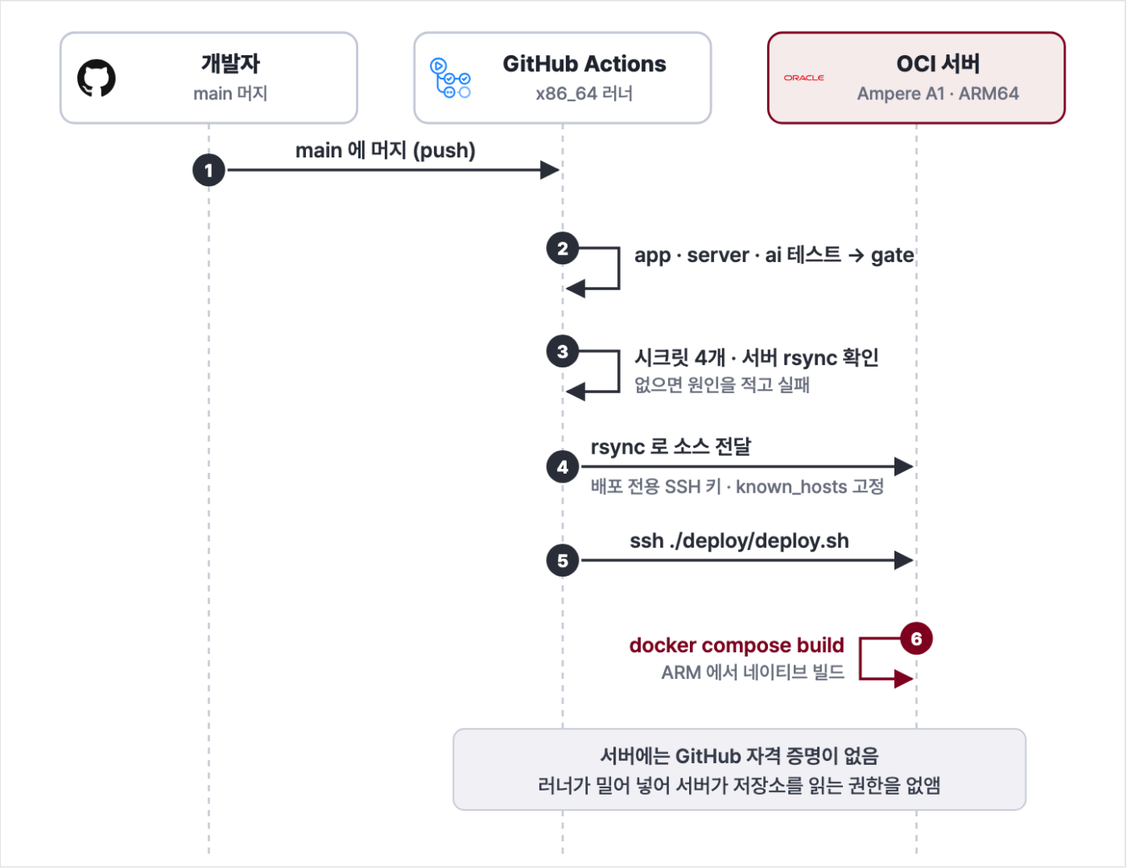
해결 서버에 설치하고, 워크플로가 배포 전에 시크릿 4개와 서버의 rsync 유무를 먼저 검사해 없으면 원인을 적고 실패하게 했습니다

옮긴 데이터 대조

이관 → 출시

검증 users · todos · archive_items 건수와 파일 저장소 목록을 이전 서버와 대조했습니다

결과 **2026.08.13 이관을 끝내고 2026.08.19 App Store에 출시했고, 서버가 바뀐 뒤에도 테스트를 통과한 코드만 배포됩니다**



PROJECT 02 — MODI

'테스트 통과 후 배포'를 다시 세우고, 끊기지 않게

배포 중 보낸 요청 1,195건
5xx 0건 (2026.08.14)

테스트와 배포를 한 워크플로로 묶기

필수 체크 = gate 하나

상황 Jenkins는 한 파이프라인에서 테스트 뒤에 배포가 이어졌지만, Actions에서 워크플로를 나누면 배포가 테스트 결과를 모른 채 push만 보고 돕니다

해결 `changes → app · server · ai → gate → deploy` 를 한 파일의 `needs` 로 묶고, 바뀐 영역만 테스트하되 건너뛴 잡은 통과로 보도록 `always() && !failure() && !cancelled()`

판단 브랜치 보호의 필수 체크는 gate 하나만 걸었습니다. 건너뛴 잡을 필수로 걸면 PR이 끝없이 기다립니다

결과 **첫 main 배포에서 테스트가 건너뛰어진 채 배포된 것을 실행 기록으로 찾아 비교 기준 브랜치를 고쳤습니다**

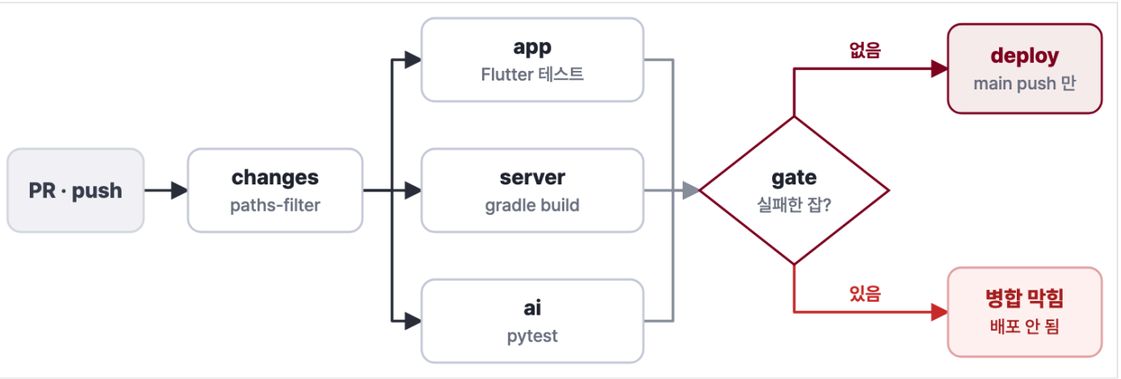
서버 한 대에서 끊기지 않게 교체

1,195요청 · 5xx 0건

상황 서버 한 대에서 컨테이너를 내렸다 올리면 그 사이 요청이 실패합니다

해결 멈춰 있던 색을 띄워 healthy가 된 뒤 활성 파일 한 줄을 바꿔 `caddy reload`, 옛 색은 graceful 종료. 실패하면 아무것도 전환하지 않습니다

결과 **배포 중 보낸 요청 1,195건 중 5xx 0건. CI를 서버 밖으로 빼서 예전 Jenkins 컨테이너가 쥐던 `docker.sock` 권한도 없앴습니다**





PROJECT 03

여신보좌관

기업 여신 심사를 돕는 AI 서비스

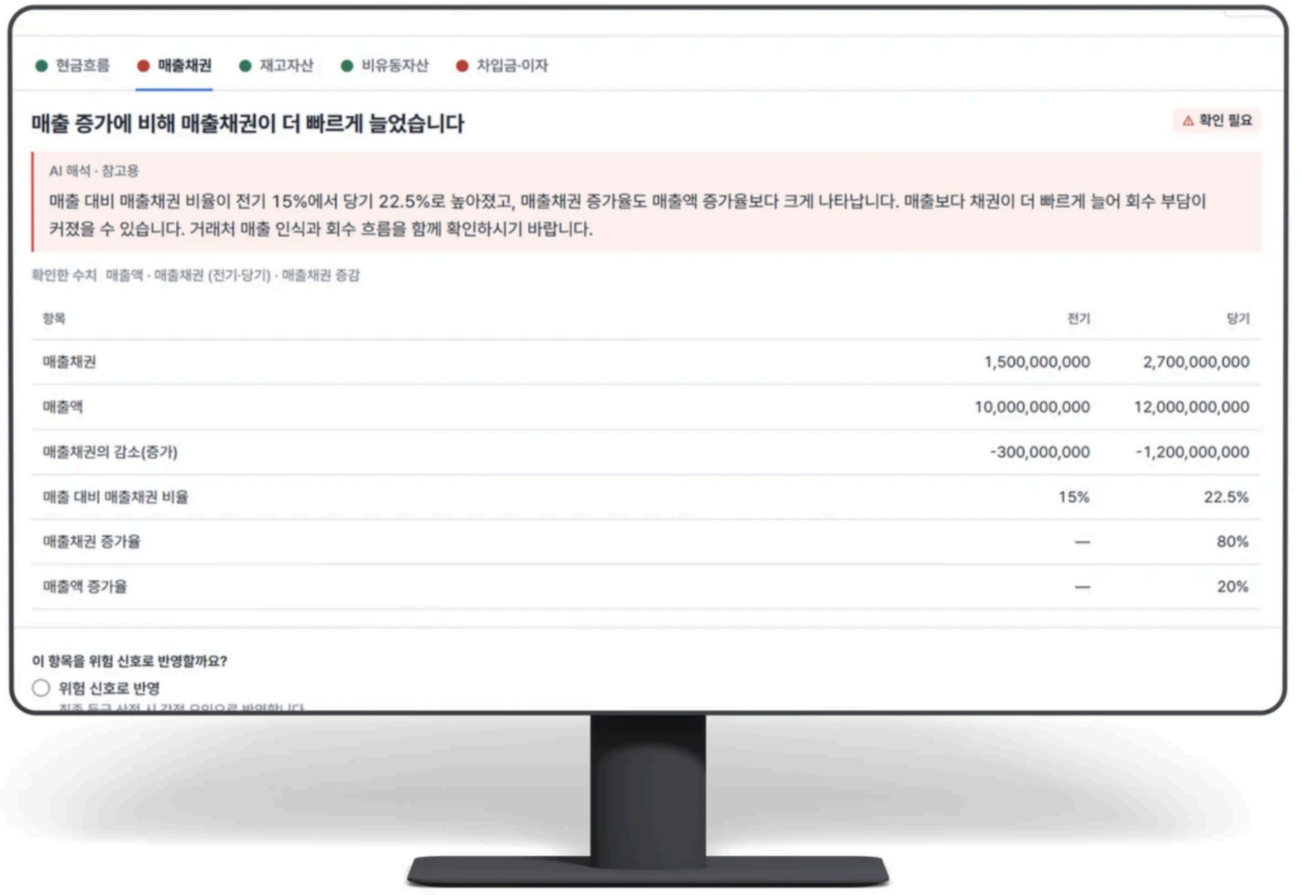
2026.08.27 – 2026.09.29 · SSAFY 2학기 특화 프로젝트

SERVICE

여보

기업여신 AI 업무 보조 서비스

서류 검토와 위험 탐지는 빠르게,
최종 판단은 기업 여신 담당자가.



PROJECT 03 — 여신보좌관

기업 여신 심사를 돕는 AI 서비스

2026.08.27 – 2026.09.29

SSAFY 2학기 특화 프로젝트 · 인프라 담당

설명

은행 여신 심사 담당자가 법인 서류를 심사할 때 판독 · 판정 · 점수 · 한도 · 심사의견 초고까지 돕는 서비스

역할

인프라 · 인증 · 감사 로그

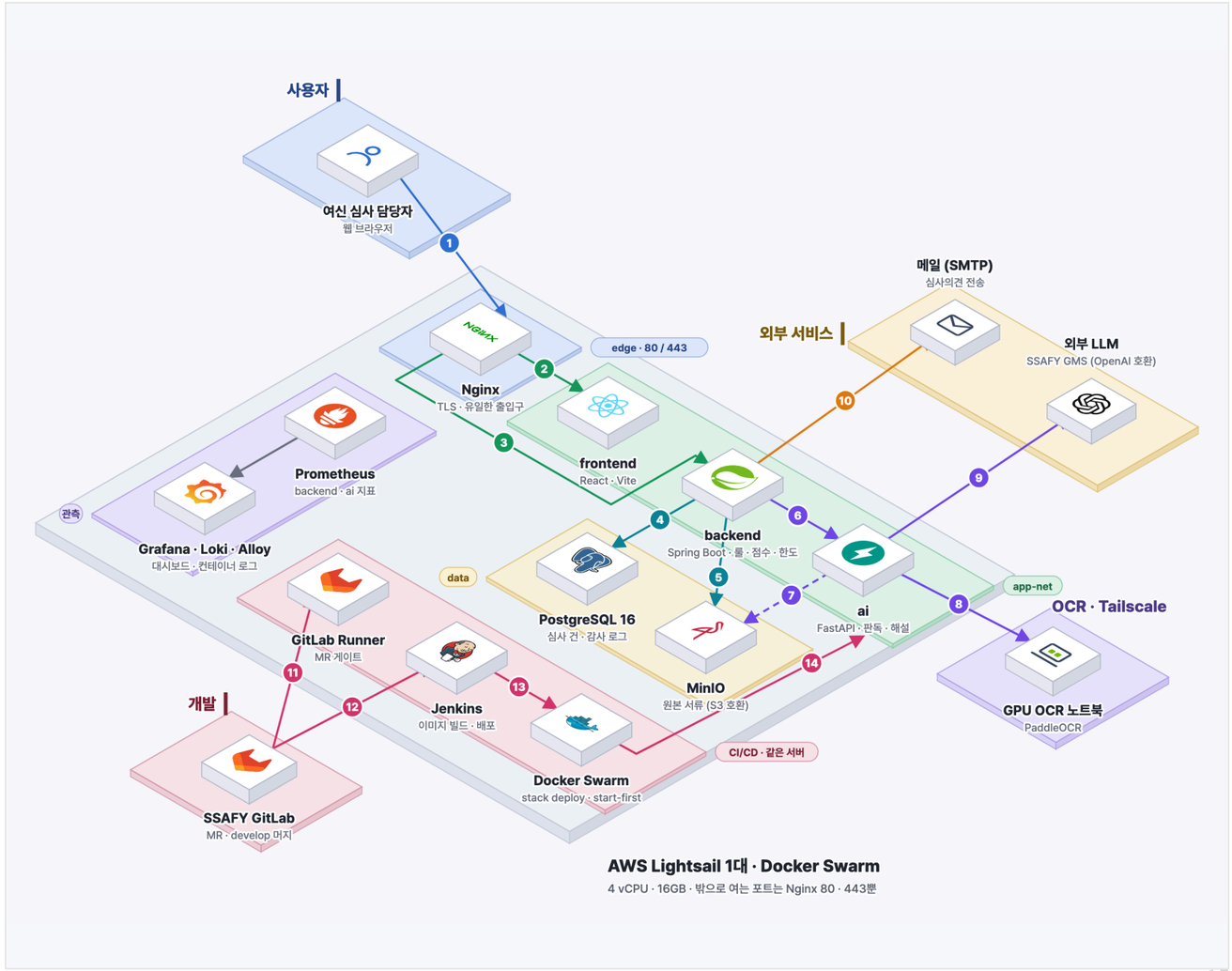
11 개	**80 · 443**	**20** / 20행
Lightsail 1대에 올린 Swarm 서비스	밖으로 여는 포트 Nginx 하나만	동시 20스레드 감사 로그 추가 체인 검증 통과

주요 구현

- DB · 파일 저장소 · 의존성을 Docker 이미지로 팀에 제공하고, develop 머지는 Jenkins가 자동 배포
- Docker Swarm start-first — 새 컨테이너가 healthy가 된 뒤 옛 컨테이너를 내림
- MR마다 포맷 · 린트 · 테스트 · 빌드 · 시크릿 스캔 · compose · nginx 설정을 검사하는 GitLab CI 게이트
- 지표는 Prometheus, 컨테이너 로그는 Alloy → Loki, 대시보드는 Grafana
- 첫 자동 배포의 502와 설정 미반영을 로그 · inode로 진단
- 감사 로그 append-only 해시 체인, 런타임 DB 계정은 audit_log에 SELECT · INSERT만

AWS Lightsail | Docker Swarm | Jenkins | GitLab CI | Nginx | Prometheus | Loki | Grafana | Tailscale | PostgreSQL

github.com/kimzoo5676-del/loanreview



PROJECT 03 — 여신보좌관

배포 성공 표시 너머의 실제 상태 확인

첫 자동 배포의 두 장애
로그와 inode를 근거로

① 첫 자동 배포가 헬스체크에서 502

DNS 다시 묻기

상황 backend 컨테이너는 172.18.0.8에 살아 있었지만, Nginx 오류 로그에는 172.18.0.9로 보낸 요청이 남아 있었습니다

원인 Nginx는 기동할 때 upstream 이름을 IP로 한 번 바꿔 계속 씁니다. 배포로 새로 뜬 backend는 새 IP를 받습니다

해결 `resolver 127.0.0.11 valid=10s` 와 `proxy_pass` 변수로, 10초가 지난 요청에서 Docker DNS에 다시 묻게 했습니다

② 다음 배포는 성공인데 설정이 그대로

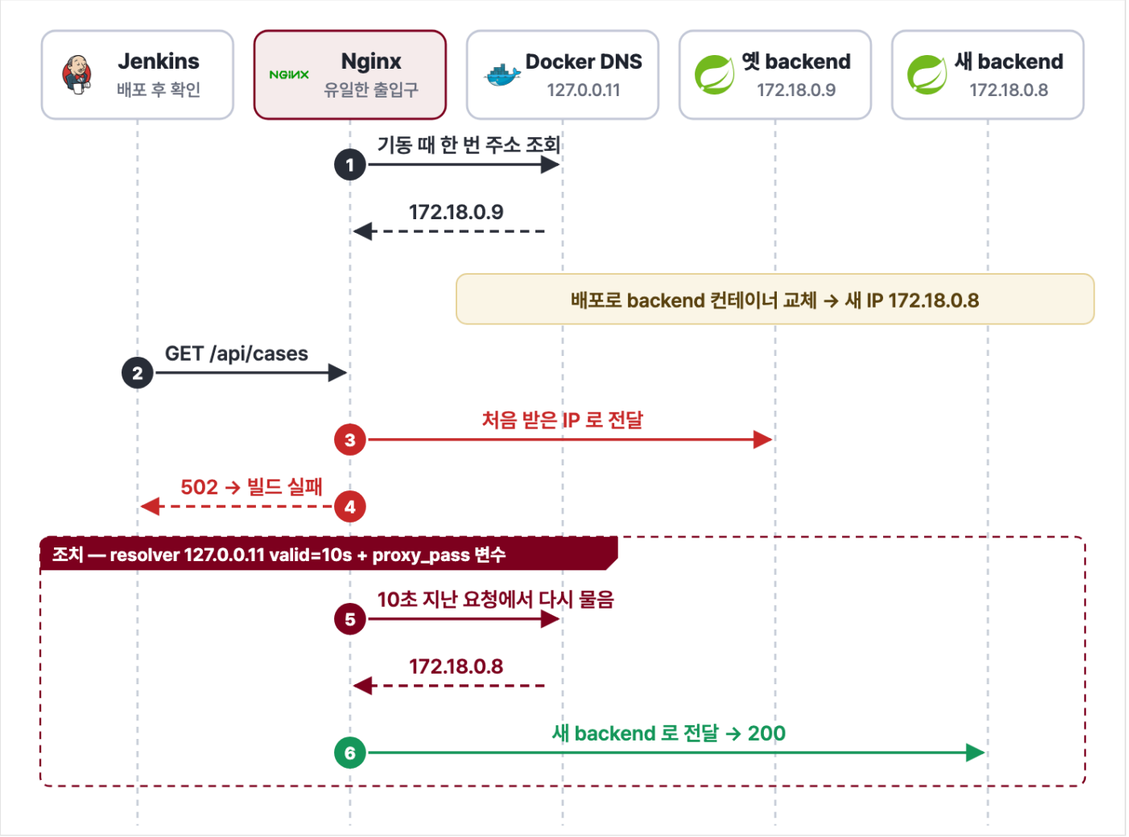
디렉터리 마운트로

상황 서버 폴더와 컨테이너 안에서 같은 설정 파일의 inode와 크기를 비교하자 서로 달랐습니다 — 서버 1112487 · 2371B, 컨테이너 1088149 · 1694B

원인 파일 하나만 바인드 마운트하면 그 순간의 inode에 묶이는데, git은 파일을 지우고 새로 써서 inode가 바뀝니다. 리눅스에서 재현해 서버 inode만 2263971 → 2263952로 바뀌는 것을 확인했습니다

해결 설정 디렉터리를 마운트해 컨테이너가 열 때마다 이름으로 찾아가게 했습니다

결과 **같은 조사에서 로컬 설정 검사가 경로 변환 때문에 빈 폴더를 검사하던 것도 찾아, 서버의 실제 설정으로 검사를 옮겼습니다**



여신보좌관 회고

전에는 인프라를 서버를 띄우고 배포하는 일로만 봤습니다. 이번에는 팀원이 CI를 기다리는 시간과, 배포가 정말 성공했는지 확인하는 기준까지 인프라가 책임질 일이라는 걸 배웠습니다.

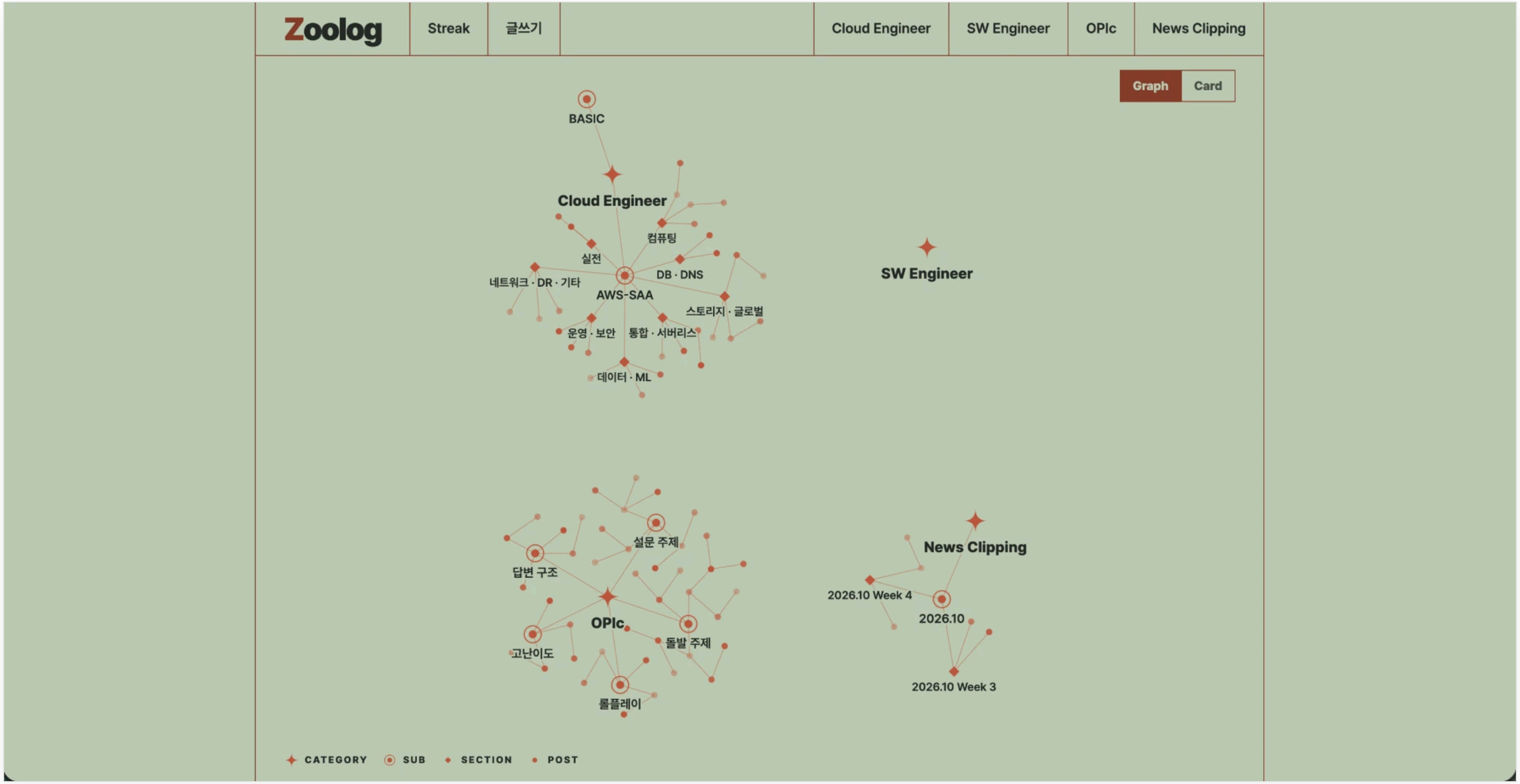
프로젝트 README「프로젝트 후기 및 느낀 점」중

PROJECT 04

Zoolog

공부 글을 그래프로 쌓는 학습 블로그

2026.10 – 운영 중 · zoolog.site



PROJECT 04 — **ZOOLOG**

공부 글을 그래프로 쌓는 학습 블로그

2026.10 – 운영 중 · 1인 개발

zoolog.site

설명

공부 글을 주제 3단계로 정리해 별자리 그래프와 주제별 달력으로 보여 주는 학습 블로그

역할

기획 · 디자인 · 백엔드 · 프론트엔드 · 인프라

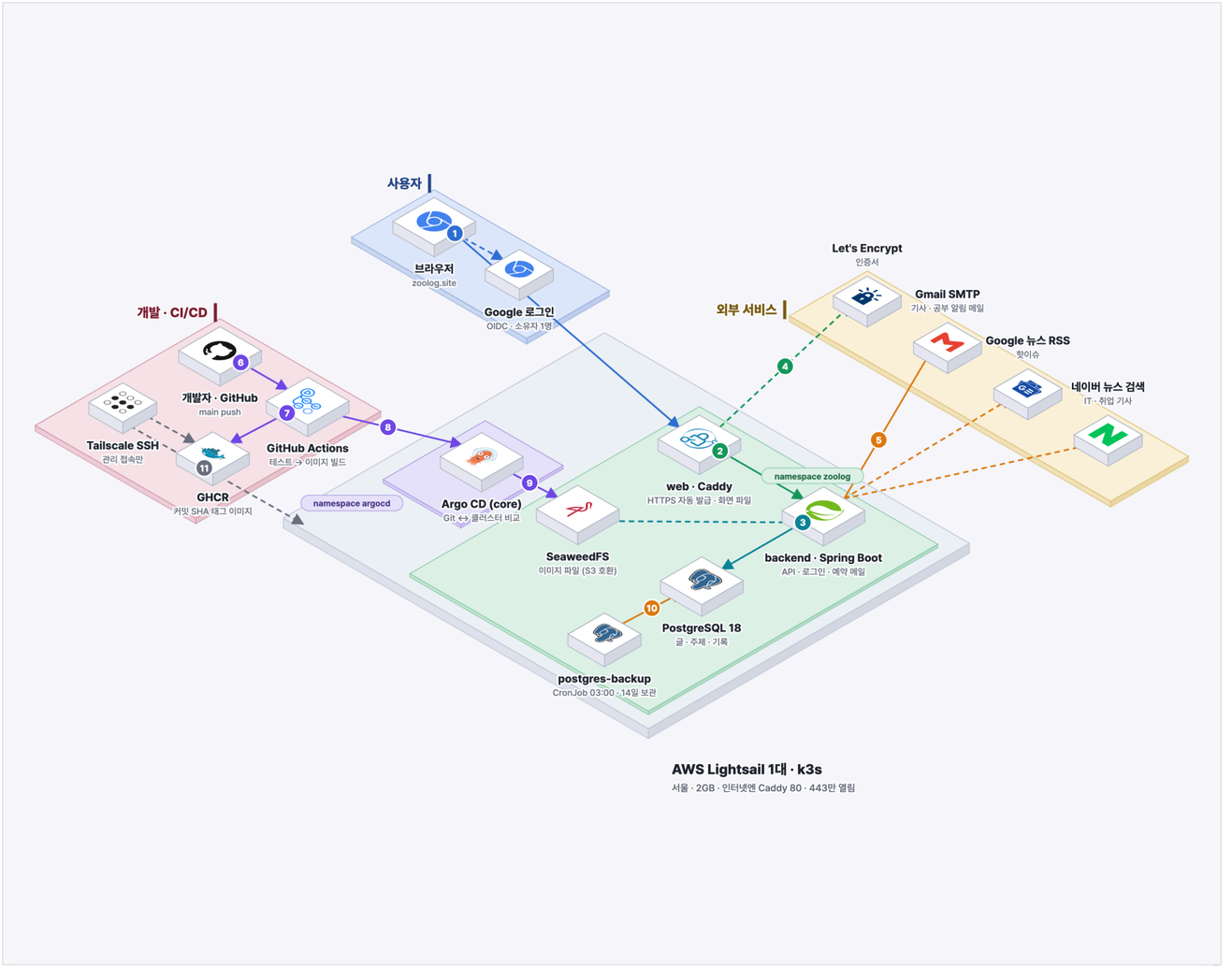
0회	**2**GB	**revert**
배포 중 화면 응답 실패 1초 간격 측정	서버 한 대 메모리에 k3s · Argo CD · 앱	되돌리기는 태그 커밋 되돌리기 한 번

주요 구현

- main push → Actions 테스트 → GHCR에 커밋 SHA 태그 → kustomization 태그 커밋 → Argo CD sync · prune · selfHeal
- 메모리를 아끼려 Argo CD를 core로 설치하고, 빠진 default AppProject를 직접 생성
- 화면 서버만 RollingUpdate(maxSurge 1 · maxUnavailable 0) + readinessProbe + preStop
- 인터넷에는 Caddy 80 · 443만, 관리는 Tailscale SSH. DB 백업 CronJob 매일 03:00 · 14일 보관
- 매일 06:00 IT · 취업 기사 메일(Google 뉴스 RSS · 네이버 뉴스 검색)

AWS Lightsail | k3s | Argo CD | Kustomize | GitHub Actions | GHCR | Caddy | Tailscale | Spring Boot | PostgreSQL

zoolog.site github.com/kimzoo5676-del/Zoolog



PROJECT 04 — ZOOLOG

메모리 2GB 서버 한 대에서 GitOps

사람은 서버에 명령을 치지 않음
Git = 서버 상태

Git에 적힌 대로 클러스터를 맞추기

되돌리기 = git revert

상황 서버에 접속해 명령으로 배포하면, 서버 상태가 Git과 어긋나도 알 수 없습니다

해결 Actions가 이미지를 커밋 SHA 태그로 GHCR에 올리고 kustomization의 태그를 커밋하면, Argo CD가 Git과 클러스터를 비교해 맞춥니다(sync · prune · selfHeal)

결과 **되돌리기는 태그 커밋을 `git revert` 한 번 — Git과 서버가 늘 같은 상태를 가리킵니다**

Argo CD core 설치 뒤 앱이 Unknown

원인을 찾아 직접 생성

상황 2GB 서버라 메모리를 아끼려 Argo CD를 core(API 서버 · 화면 없이)로 설치하자 앱이 Unknown에 멈췄습니다

원인 상태 메시지 `project default which does not exist` — core 설치에는 default 프로젝트를 만드는 argocd-server가 없습니다

해결 default AppProject를 직접 만들고, 설치 절차 문서에 추가했습니다

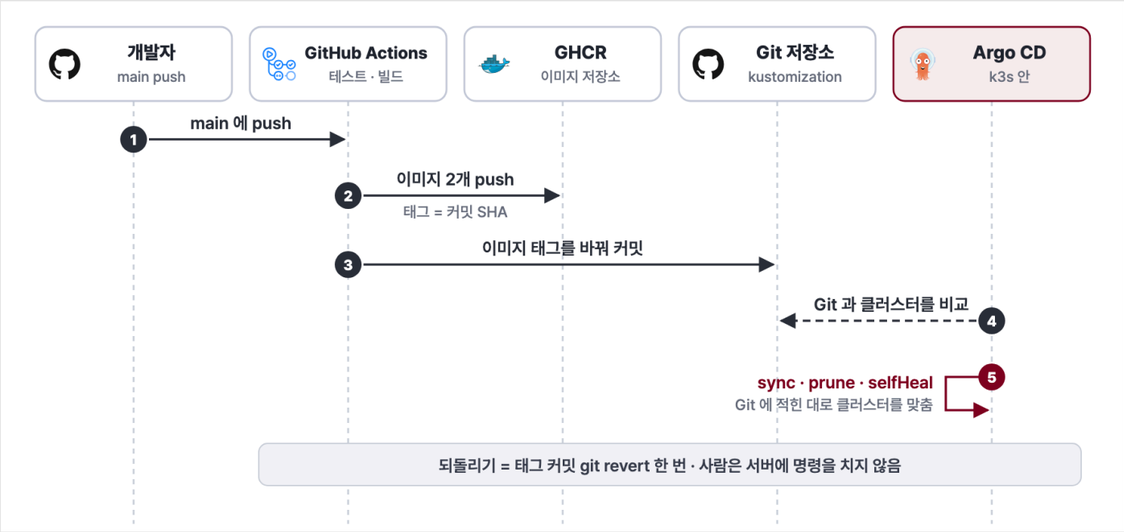
화면 서버 무중단 교체

응답 실패 0회

해결 화면 서버(Caddy)를 RollingUpdate(maxSurge 1 · maxUnavailable 0)로 바꾸고 readinessProbe와 preStop을 붙였습니다

결과 **배포 중 1초 간격 측정에서 화면 응답 실패 0회**

남은 일 백엔드는 2GB 메모리 한계로 Recreate(약 30초 API 중단) — 메모리 사용량을 보고 다음 단계를 정하는 중입니다



PROJECT 05

OURWISH

사회초년생을 위한 예·적금 추천 서비스

2026.05 – 2026.06 · 1학기 관통 프로젝트 최우수상



PROJECT 05 — OURWISH

사회초년생을 위한 예·적금 추천 서비스

2026.05 – 2026.06 · 팀 프로젝트

SSAFY 1학기 관통 프로젝트 최우수상 (1위)

역할

Backend · Infra · AI — 추천 API, DB 스키마, 데이터 적재, 배포

338→115개	766건	1위
우대조건 있음으로 잡힌 적금 우대율이 적힌 줄만 인정	금감원 수집 + AI 가공 예·적금 상품	1학기 관통 프로젝트 최우수상

주요 구현

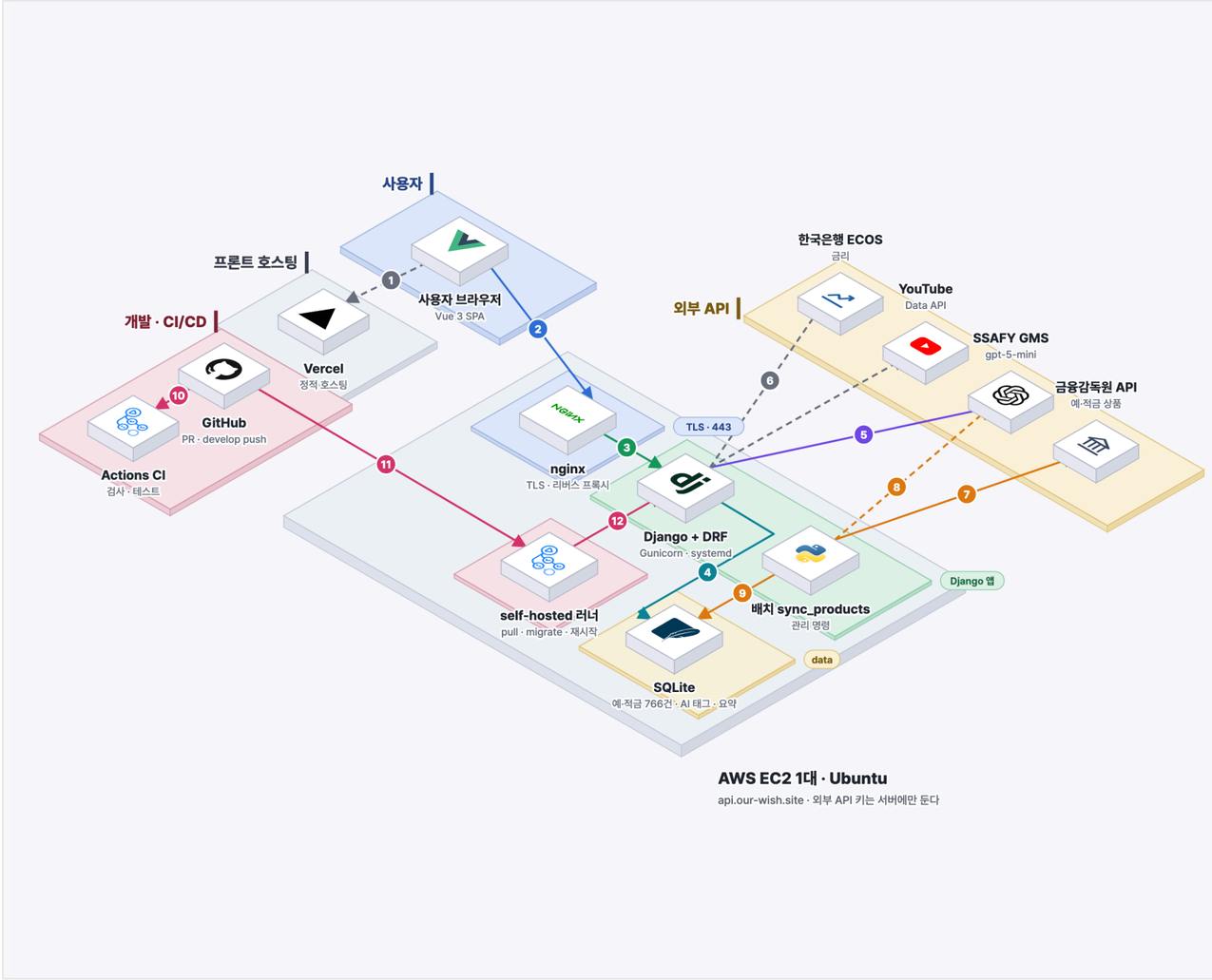
- 서버에 접속해 하던 수동 배포를 GitHub Actions(EC2 self-hosted 러너)로 — 발표 주에는 버튼으로, 이후 develop push 자동 배포. 헬스 체크가 실패하면 직전 커밋으로 롤백
- 우대조건 원문을 공통 조건 태그로 정형화하는 전처리 — 응답 JSON 고정, 실패 상품은 건너뛰고 미처리분만 재처리
- 남은 정합성 문제 때문에 우대금리 합산 대신 금감원 기본 · 최고 금리를 그대로 사용
- 추천 API는 미리 저장한 조건 · 태그로 거르고, 요청 때 LLM을 부르지 않음
- 외부 API 키는 서버에만 — 금리 · 영상 검색은 Django가 대신 호출

OURWISH 회고

가정하지 말고 실제 데이터를 직접 확인해야 한다는 것을 배웠습니다. 모든 우대조건을 빠짐없이 구조화하겠다는 욕심을 버리고, 데이터를 직접 확인하며 가장 빈번한 보편 우대조건만 추려 표준 태그로 정했습니다.

프로젝트 README「프로젝트 후기 및 느낀 점」 중

our-wish.site github.com/Our-Wish/ourwish



ENGINEERING · STUDY

Additional

SSAFY RACE · Zoolog 학습 기록

자율주행 시뮬레이션 경진대회 – SSAFY RACE

2026.05 – 2026.06 · 3인

278팀 중 2위 (777명 참가)

설명

실시간 차량 · 도로 센싱 정보로 조향과 가감속을 제어해 트랙을 완주하는 자율주행 알고리즘

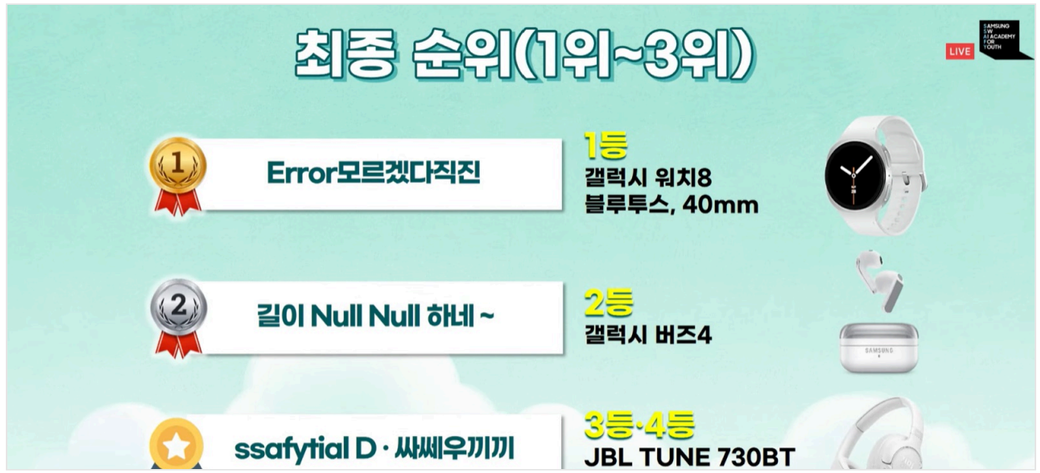
역할 · 환경

주행 알고리즘 — Python · Microsoft AirSim · Driving API

AI로 가설을 세우고 기록으로 가르기

- 목표를 '완주를 보장하면서 랩타임을 줄인다'로 정하고 대회 규칙 · API 명세 · 제어 변수를 AI에 전달
- 가설마다 변수 하나만 바꿔 새 파일에서 실행, 랩타임과 완주 여부로 채택 · 기각
- 장애물 개수마다 전략을 나눈 다섯 버전은 모두 기록이 나빠지거나 완주 실패 → 공개된 14기 4등 코드의 단일 규칙(가장 가까운 빈 공간)을 베이스로 선택하고 그 위에서 튜닝
- 코너 안쪽 주행선은 Speed 2:44 → 2:27이지만 Germany에서는 이탈 → 맵마다 베이스를 나눔
- "최고 기록만 남으니 마음껏 제출해도 된다"는 AI 답을 대회 규정(마지막 제출 코드로 순위)으로 바로잡고, 정규 리그에는 실험 코드를 내지 않음
- 채택 · 기각 이유와 다시 시도하지 않을 방향을 인계 문서에 남겨 다음 세션의 AI가 그 기록에서 출발





공부한 것을 쌓고, 프로젝트에 적용하기

직접 운영하는 Zoolog에 AWS 자격 범위를 장마다 정리하고, **아침 기사 메일로 받은 IT 기사를 읽고 제 생각으로 정리**합니다.

Cloud Engineer › AWS-SAA

2026.10.16 IAM
최소 권한 정책 · Condition 키 · 루트 키 삭제 → NEW:LEARN apply 권한 설계에 적용

2026.10.19 EC2
인스턴스 유형 · 보안 그룹 · 구매 옵션(On-Demand · Reserved · Spot)

2026.10.20 EC2 심화
Private · Public · Elastic IP, 배치 그룹, ENI · ENA · EFA, Hibernate

News Clipping

2026.10.15 AI 시대의 병목은 GPU가 아니라 데이터 — 넷앱 차세대 스토리지

2026.10.19 AWS, 로봇용 오픈소스 '피지컬 AI 툴체인' 출시

2026.10.20 KT, 2026 AID-X Day — AI 혁신 성과 공개

2026.10.21 AI 자기복제 앞당겨지나…앤트로픽 잇단 경고

> 엔지니어에게 가장 중요한 것은 어디까지 AI에게 맡기고 어디서부터 사람이 직접 판단해야 하는지 그 경계를 설정하는 능력이라는 생각이 들었다.
>
> Zoolog 기사 스크랩 「KT, 2026 AID-X Day」 중 내 생각 정리



Zoolog 뉴스 스크랩 — 기사 요약 뒤에 「내 생각 정리」를 붙인다

THANK YOU

감사합니다

배포를 자동화하고, 실제 상태를 재고,
인프라를 코드와 권한으로 다루는 클라우드·인프라 엔지니어 김주우입니다.

Email	kimzoo5676@gmail.com
GitHub	github.com/kimzoo5676-del
Blog	zoolog.site
Web	kimzoo5676-del.github.io